

Technical Report - Assginment11

컴퓨터공학과 2317022 이다영

cf) 실험 결과과 코드를 실행한 환경에 대한 정보는 레포트 마지막 "실험결과" 부분에 작성되어 있습니다.

UNet.py

UNet.py는 UNet을 구현한다. UNet은 encoder, decoder와 이들을 잇는 skip connection으로 구성된 네트워크이며, encoder는 convolution과 max pooling으로 다운샘플링을 수행하고 decoder는 up-convolution과 convolution으로 업샘플링을 수행한다.

1. conv(in_channels, out_channels)

```
def conv(in_channels, out_channels):  
    return nn.Sequential(  
        nn.Conv2d(in_channels, out_channels, 3, padding=1),  
        nn.BatchNorm2d(out_channels),  
        nn.ReLU(inplace=True),  
        nn.Conv2d(out_channels, out_channels, 3, padding=1),  
        nn.BatchNorm2d(out_channels),  
        nn.ReLU(inplace=True)  
    )
```

conv()는 UNet에서 사용할 convolution 블록을 [3×3 Conv] → [BN] → [ReLU] → [3×3 Conv] → [BN] → [ReLU] 구조로 정의한다.

2. class Unet(nn.Module)

Unet 클래스에서는 UNet 전체 구조를 정의하고 forward() 함수를 구현해 순전파를 정의한다.

2.1 init(self, in_channels, out_channels)

```
self.convDown1 = conv(in_channels, 64)  
self.convDown2 = conv(64, 128)  
self.convDown3 = conv(128, 256)  
self.convDown4 = conv(256, 512)  
self.convDown5 = conv(512, 1024)  
  
self.maxpool = nn.MaxPool2d(2, stride=2)  
self.upsample = nn.Upsample(scale_factor=2, mode='bilinear', align_corners=True)  
  
self.convUp4 = conv(1536, 512) # 1024 + 512 = 1536  
self.convUp3 = conv(768, 256) # 512 + 256 = 768  
self.convUp2 = conv(384, 128) # 256 + 128 = 384  
self.convUp1 = conv(192, 64) # 128 + 64 = 192
```

```
self.convUp_fin = nn.Conv2d(64, out_channels, 1)
```

encoder는 convDown1부터 convDown5까지 구성되며 채널 수가 $64 \rightarrow 128 \rightarrow 256 \rightarrow 512 \rightarrow 1024$ 로 2배씩 증가한다. MaxPool2d로 stride=2 다운샘플링하고, decoder에서 사용할 Upsample을 scale_factor=2로 정의한다.

decoder의 convUp 레이어는 skip connection으로 encoder feature map을 concat하므로 입력 채널이 두 배가 된다. 예를 들어 convUp4는 1024채널에 convDown4의 512채널을 concat한 1536채널을 입력받아 512채널을 출력한다. 마지막으로 convUp_fin에서 1x1 conv로 최종 클래스 수만큼 채널을 출력한다.

2.2 forward(self, x)

```
conv1 = self.convDown1(x)
x = self.maxpool(conv1)
conv2 = self.convDown2(x)
x = self.maxpool(conv2)
conv3 = self.convDown3(x)
x = self.maxpool(conv3)
conv4 = self.convDown4(x)
x = self.maxpool(conv4)
conv5 = self.convDown5(x)

x = self.upsample(conv5)
x = torch.cat([x, conv4], dim=1)
x = self.convUp4(x)
x = self.upsample(x)
x = torch.cat([x, conv3], dim=1)
x = self.convUp3(x)
x = self.upsample(x)
x = torch.cat([x, conv2], dim=1)
x = self.convUp2(x)
x = self.upsample(x)
x = torch.cat([x, conv1], dim=1)
x = self.convUp1(x)

out = self.convUp_fin(x)
return out
```

encoder에서는 convDown과 maxpool을 반복하며 feature map을 다운샘플링한다. 각 단계의 결과를 conv1~conv4로 저장해 decoder에서 skip connection으로 활용한다. decoder에서는 upsample 후 torch.cat()으로 대응하는 encoder feature map과 concat하고 convUp을 적용하는 과정을 반복한다. 마지막으로 convUp_fin에서 1x1 conv로 최종 segmentation map을 출력한다.

resnet_encoder_unet.py

resnet_encoder_unet.py에서는 ResNet-50을 encoder로 사용하는 UNet을 구현한다.

1. ResidualBlock

```

if self.downsample:
    self.layer = nn.Sequential(
        conv1x1(in_channels, middle_channels, stride=2, padding=0),
        conv3x3(middle_channels, middle_channels, stride=1, padding=1),
        conv1x1(middle_channels, out_channels, stride=1, padding=0)
    )
    self.downsize = conv1x1(in_channels, out_channels, 2, 0)
else:
    self.layer = nn.Sequential(
        conv1x1(in_channels, middle_channels, stride=1, padding=0),
        conv3x3(middle_channels, middle_channels, stride=1, padding=1),
        conv1x1(middle_channels, out_channels, stride=1, padding=0)
    )
    self.make_equal_channel = conv1x1(in_channels, out_channels, 1, 0)

```

bottleneck residual block을 구현한다. downsample=True인 경우 첫 번째 1x1 conv의 stride=2로 spatial resolution을 절반으로 줄이며, shortcut connection도 downsize로 동일하게 맞춘다. forward()에서는 이전 과제와 달리 `activation(out + x)` 로 ReLU를 residual 합산 이후에 적용한다.

2. UNetWithResnet50Encoder — init 및 forward

```

self.layer1 = nn.Sequential(
    nn.Conv2d(3, 64, kernel_size=7, stride=2, padding=3),
    nn.BatchNorm2d(64),
    nn.ReLU(inplace=True)
)
self.pool = nn.MaxPool2d(3, 2, 1, return_indices=True)

self.layer2 = nn.Sequential(
    ResidualBlock(64, 64, 256, False),
    ResidualBlock(256, 64, 256, False),
    ResidualBlock(256, 64, 256, True)
)
self.layer3 = nn.Sequential(
    ResidualBlock(256, 128, 512, False),
    ResidualBlock(512, 128, 512, False),
    ResidualBlock(512, 128, 512, False),
    ResidualBlock(512, 128, 512, False)
)

```

ResNet-50의 layer1~layer3을 encoder로 사용한다. layer2의 마지막 블록과 layer3의 마지막 블록은 downsample=True로 설정해 각 단계에서 spatial resolution을 절반으로 줄인다.

forward()는 아래와 같다:

```

out1 = self.layer1(x)
out1, indices = self.pool(out1)
out2 = self.layer2(out1)

```

```

out3 = self.layer3(out2)
x = self.bridge(out3)
x = self.UpConv1(x)
x = torch.cat([x, out2], dim=1) # skip connection: out2(256ch)와 concat → 512ch
x = self.UnetConv1(x)
x = self.upconv2_1(x, output_size=torch.Size([x.size(0), 256, 64, 64]))
x = self.upconv2_2(x)
x = torch.cat([x, out1], dim=1) # skip connection: out1(64ch)와 concat → 128ch
x = self.UnetConv2_2(x, output_size=torch.Size([x.size(0), 128, 128, 128]))
x = self.UnetConv2_2(x, output_size=torch.Size([x.size(0), 128, 256, 256]))
x = self.UnetConv2_3(x)
x = self.UnetConv3(x)
return x

```

out1, out2, out3을 skip connection으로 활용해 UNet decoder와 연결한다. torch.cat()으로 encoder feature map을 concat하며 ConvTranspose2d로 업샘플링한다. UnetConv2_2를 두 번 적용해 32×32 → 128×128 → 256×256으로 해상도를 복원하여 입력과 동일한 256×256 출력을 생성한다.

modules.py

1. train_model()

```

optimizer.zero_grad()
outputs = model(inputs)
loss = criterion(outputs, labels)
loss.backward()
optimizer.step()
scheduler.step()

```

trainloader를 순회하며 gradient 초기화 → 순전파 → 손실 계산 → 역전파 → 파라미터 갱신 → 스케줄러 갱신 순으로 학습을 수행한다.

2. get_loss_train() 및 val_model()

```

outputs = model(inputs)
loss = criterion(outputs, labels)

```

두 함수 모두 model.eval() 및 torch.no_grad() 하에서 순전파와 손실 계산만 수행한다. 파라미터 업데이트는 하지 않는다. val_model()에서는 추가로 cls_invert 디크셔너리를 사용해 예측 결과를 RGB 이미지로 변환하고 .png 파일로 저장한다.

```

temp_rgb[j, k] = cls_invert[int(temp[j, k])] # 예측값 → RGB
temp_label[j, k] = cls_invert[int(temp_l[j, k])] # 정답 → RGB

```

main.py

```
model = UNetWithResnet50Encoder(n_classes=22)
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=learning_rate)
scheduler = StepLR(optimizer, step_size=4, gamma=0.1)
```

모델은 UNetWithResnet50Encoder로 초기화하며 손실 함수는 CrossEntropyLoss, optimizer는 Adam(lr=0.001), 스케줄러는 StepLR(step_size=4, gamma=0.1)을 사용한다.

```
model.load_state_dict(torch.load(pretrained_path, map_location=device))
epochs = 1
```

과제함에서 제공받은 pretrained checkpoint `resnet_encoder_unet.pth`를 로드하며, 이 경우 epochs=1로 설정되어 1 epoch만 학습한다. epoch가 4의 배수일 때마다 `torch.save(model.state_dict(), savepath2)`로 checkpoint를 저장한다.

실험 결과

Google Colab GPU(T4) 환경에서 교수님 제공 pretrained checkpoint를 로드하여 1 epoch 학습한 결과는 아래와 같다.

두 가지 경우를 모두 실험해 보기 위해서, unet, resnet-encoder-unet 두가지에 대해 각각 main 스크립들을 만들었다.

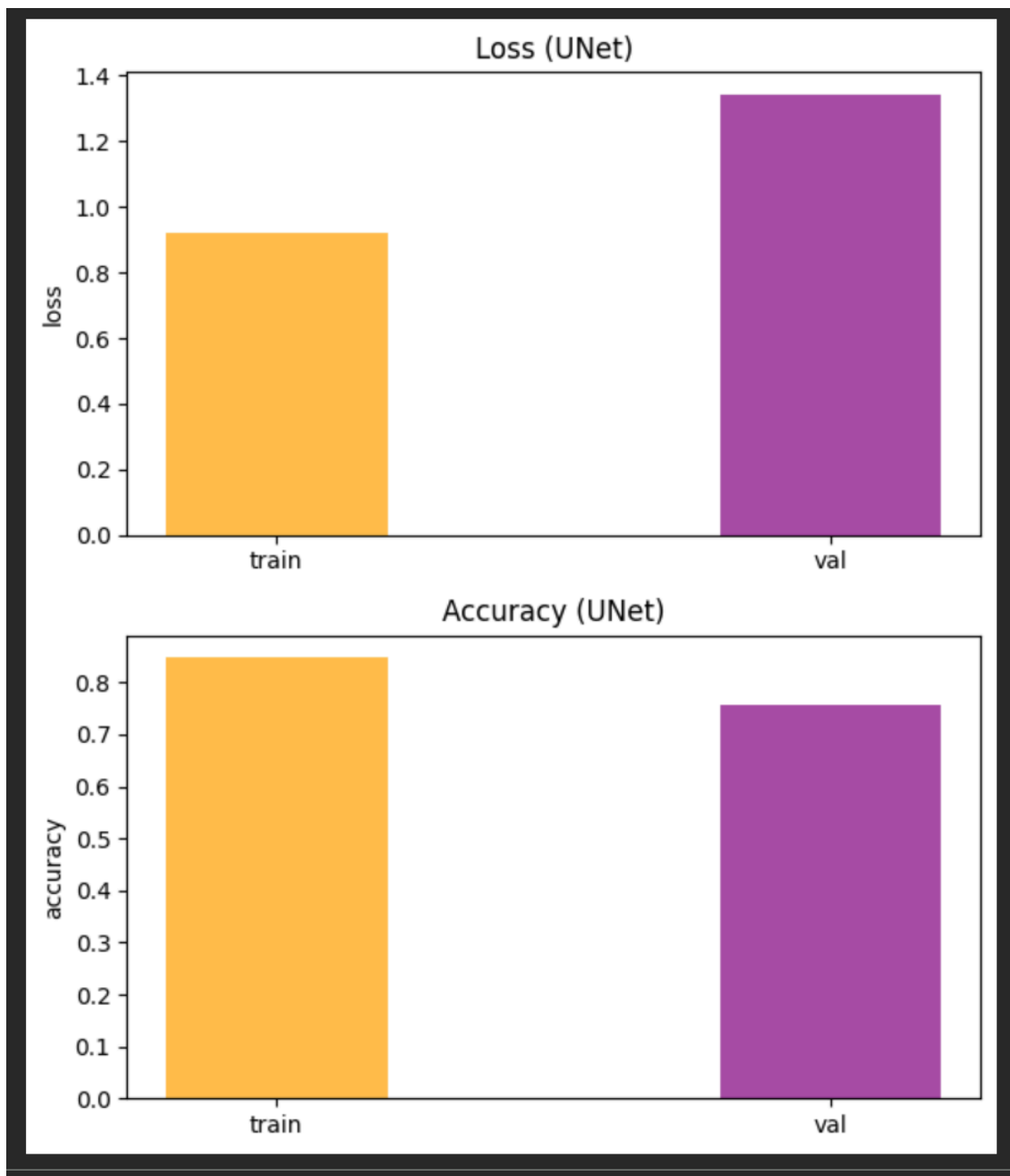
`main_Unet.py`

UNet

Google Colab GPU(T4) 환경에서 제공해주신 pretrained checkpoint(`UNet_trained_model.pth`)를 로드하여 1 epoch 학습하였다.

	Loss	Accuracy
Train	0.9192	84.79%
Validation	1.3430	75.68%

```
epoch 1 train loss : 0.9192403795784467 train acc : 0.8478977012159221
epoch 1 val loss : 1.3429840948130634 val acc : 0.7567747391022004
Finish Training
Fin
```

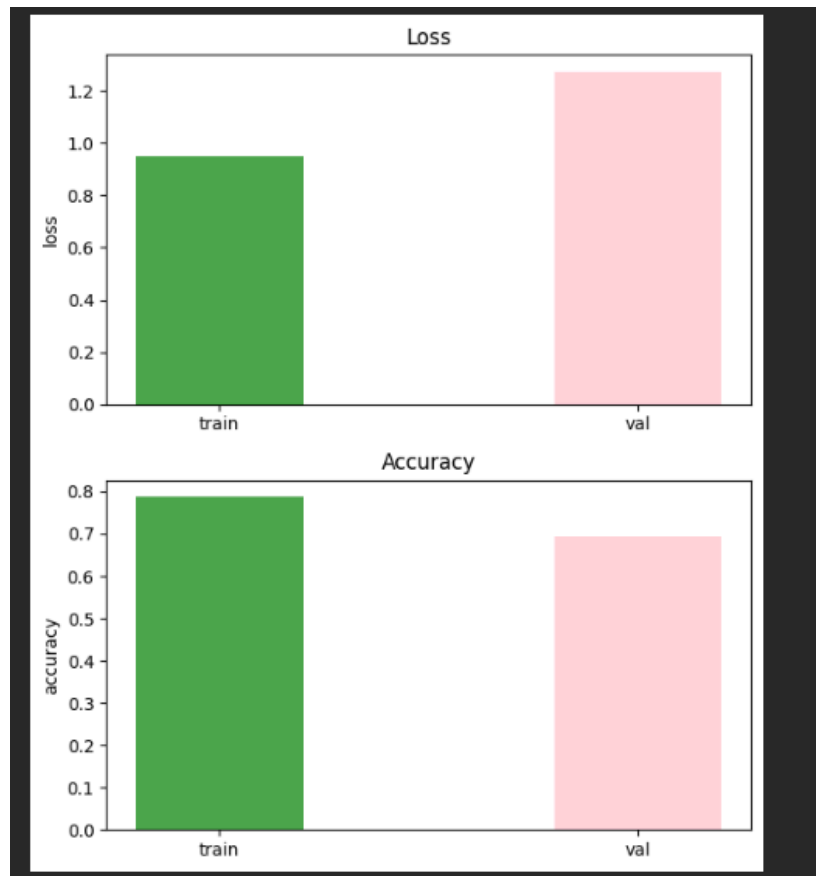




main_resnet.py

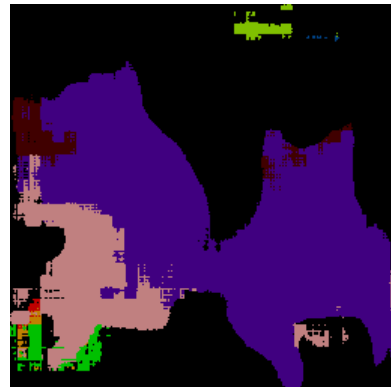
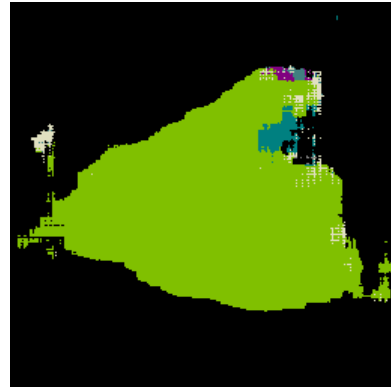
	Loss	Accuracy
Train	0.9471	78.73%
Validation	1.2735	69.36%

```
epoch 1 train loss : 0.9471331361221941 train acc : 0.7872691641412072
epoch 1 val loss : 1.2735049144641772 val acc : 0.6935989276782887
Saved checkpoint: ./output/model2026-06-14(09-50)/0.pth
Finish Training
Fin
```

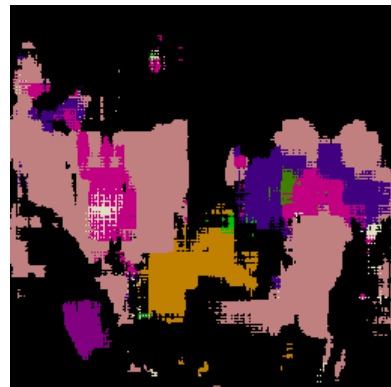


결과 이미지들은 아래와 같다. 각 쌍별로 왼쪽이 label, 오른쪽이 result 이다.





[#533]



결과 분석

두 모델 모두 pretrained checkpoint를 활용하여 1 epoch만 학습하였음에도 준수한 성능을 보였다.

	Train Loss	Train Acc	Val Loss	Val Acc
ResNet-encoder-UNet	0.9471	78.73%	1.2735	69.36%
UNet	0.9192	84.79%	1.3430	75.68%

두 모델 모두 validation loss가 train loss보다 높고 validation accuracy가 train accuracy보다 낮은 경향을 보이는데, 이는 모델이 학습 데이터에 어느 정도 적합되어 있기 때문으로 일반적인 현상이다.

흥미롭게도 ResNet-50을 encoder로 사용한 UNetWithResnet50Encoder보다 기본 UNet의 train accuracy(84.79%)와 val accuracy(75.68%) 모두 더 높게 나타났다. 이는 UNet pretrained checkpoint가 더 많은 epoch 동안 학습된 것이거나, 혹은 Pascal VOC 2012와 같은 비교적 단순한 segmentation 태스크에서는 더 가벼운 UNet 구조가 오히려 효과적일 수 있음을 시사한다.

segmentation 결과 이미지를 보면 두 모델 모두 객체의 전반적인 위치와 형태를 어느 정도 파악하고 있으나, 세밀한 경계 부분에서는 부정확한 예측이 나타난다. 더 많은 epoch를 학습시킬 경우 두 모델 모두 성능이 향상될 것으로 예상된다.

소감

이번 학기 마지막 과제인 Assignment 11과제는 은 강의에서 배운 UNet과 ResNet을 직접 구현하고 실제 데이터셋에 적용해보는 과제로, 이론으로만 접했던 skip connection과 encoder-decoder 구조를 코드로 구현하며 그 동작 원리를 더 깊이 이해할 수 있었다. 특히, ResNet-50을 encoder로 대체하는 과정에서 두 아키텍처가 어떻게 결합되는지 파악하는 것이 흥미로웠다. 한 학기 동안 CNN, ResNet, UNet 등 다양한 딥러닝 모델을 직접 구현하며 수업을 마무리할 수 있어 의미 있었다.